

AI AGENT 学习路线 · 学习专题 007

多 Agent 系统 学习笔记（打印版）

11 个篇章 · 7 张矢量架构图

生成日期：2026-09-19 | 由 Markdown 笔记自动排版生成

目录

01	007 · 多 Agent 系统 (Multi-Agent Systems)	导航
02	00 · 背景篇: 多 Agent 前史与全景	含架构图 ✓
03	007 · 技术篇导航 (techniques/)	
04	01 · 问题篇: 单 Agent 什么时候不够用	含架构图 ✓
05	02 · 概念篇: 核心概念精讲 (11 张概念卡)	含架构图 ✓
06	03 · 原理篇: 协作模式与通信解剖	
07	04 · 进阶篇: 编排拓扑与 A2A 协议	含架构图 ✓
08	05 · 实战篇: 手写一个多 Agent 协作 (Python 全代码)	含架构图 ✓
09	06 · 生产篇: 成本、上下文与失败模式	含架构图 ✓
10	07 · 对比篇: 框架与拓扑选型	
11	08 · 自测篇: 题库与面试题	分级题库 ✓

共 11 个篇章。

007 • 多 Agent 系统 (Multi-Agent Systems)

一句话定位：当一个 Agent 装不下、跑不动、管不清的时候，把活拆给一群角色化的 Agent——但先算清楚 15 倍 Token 的账。

这章讲什么

单 Agent 是"一个全栈工程师硬扛所有活"；多 Agent 是"组建一个小团队"。本章讲清三件事：

- 1. 什么时候该拆：单 Agent 的四大天花板（上下文装不下、工具太多、串行慢、Prompt 膨胀），以及什么时候不该拆（Anthropic 观点：能单就单，多 Agent 是双刃剑）；
- 2. 拆了怎么协作：角色、编排者、通信、handoff、拓扑（星型/流水线/层级/网状）、A2A 协议；
- 3. 怎么不翻车：失败模式（谁也不认账、信息丢失、循环踢皮球、成本失控）与防御。

贯穿案例：点价评审小组——来自中基集团 CTRM 贸易系统的真实场景。主管 Agent 接到一笔点价单的评审任务，拆解后并行派给三个 Worker（抽取员抽合同要素、风控员核风险、定价员验价格），收齐三份结构化交付物后交叉核对，汇总出具评审意见。全程对应 Java 开发者熟悉的概念：orchestrator ≈ workflow 引擎，handoff ≈ 工单转派，Schema ≈ DTO 契约，并行 ≈ `CompletableFuture.allOf`。

目录树

007-多agent系统/	
├─ README.md	← 你在这里：导航
├─ 00-背景篇-多agent前史与全景.md	← 从 1980s MAS 到 2026 A2A 入 AAIL
├─ techniques/	
│ └─ README.md	← 技术篇导航
│ └─ 01-问题篇-单agent什么时候不够用.md	
│ └─ 02-概念篇-核心概念精讲.md	← 11 张概念卡
│ └─ 03-原理篇-协作模式与通信解剖.md	
│ └─ 04-进阶篇-编排拓扑与A2A协议.md	
│ └─ 05-实战篇-手写一个双agent协作.md	← Python 全代码可跑
│ └─ 06-生产篇-成本上下文与失败模式.md	
│ └─ 07-对比篇-框架与拓扑选型.md	
│ └─ 08-自测篇-题库与面试题.md	
└─ diagrams/	← 7 张 SVG 图册
└─ 007-01-多agent演进时间线.svg	
└─ 007-02-单agent天花板与拆分信号.svg	
└─ 007-03-多agent核心概念地图.svg	
└─ 007-04-四种协作拓扑对比.svg	← 重点图
└─ 007-05-A2A与MCP互补关系.svg	
└─ 007-06-点价评审小组架构.svg	← 贯穿案例架构
└─ 007-07-多agent失败模式与防御.svg	

阅读路线

你想	去哪
快速建立全景与时间线	00-背景篇（配 007-01 时间线图）
判断自己的场景该不该上多 Agent	01-问题篇（配 007-02 信号图）
补齐术语，看懂别人在说什么	02-概念篇（配 007-03 概念地图）
理解协作与通信的底层机制	03-原理篇
吃透拓扑选型与 A2A 协议	04-进阶篇（配 007-04 / 007-05）
动手写一个能跑的协作系统	05-实战篇（配 007-06 案例架构）
生产环境防翻车	06-生产篇（配 007-07）
选框架、做决策	07-对比篇
面试 / 复盘自查	08-自测篇

与其他章节的衔接

章节	衔接点
003-上下文工程	多 Agent 的核心收益是 上下文隔离 ：每个 SubAgent 只带自己需要的上下文。003 讲 context rot（上下文腐烂）——信息越多注意力越稀释，多 Agent 正是对症的手术刀。03-原理篇会反复回指。
004-单agent设计模式	多 Agent 不是替代而是升级：ReAct、Plan-and-Execute 在每个 SubAgent 内部照用不误。先掌握单 Agent 的工具循环，再看本章如何把多个循环编排起来。
006-MCP协议	MCP 是 Agent 连接 工具/数据 的协议（纵向）；A2A 是 Agent 连接 Agent 的协议（横向）。006 吃透 MCP 的 Server/Client 模型，本章的 A2A 一节会非常顺。
009-框架实战	本章 05 实战篇手写"无框架多 Agent"，看透本质；07 对比篇给出框架选型决策树，具体框架 API 细节与完整项目见 009。

图册总览

图	一句话
007-01 演进时间线	从学术 MAS 到 2026-08 A2A 进 AAIF，9 个关键节点
007-02 天花板与拆分信号	四大天花板 + 该拆/不该拆双清单
007-03 核心概念地图	11 个概念卡环绕一张中心图
007-04 四种拓扑对比	星型/流水线/层级/网状的图解+优劣+适用（重点）
007-05 A2A 与 MCP	一个管连 Agent、一个管连工具，互补两层
007-06 点价评审小组架构	贯穿案例：主管 + 三 Worker 的星型全景

图	一句话
007-07 失败模式与防御	四大失败模式 → 四个写进代码的防御

00 • 背景篇：多 Agent 前史与全景

【本节问题】① 为什么 2023 年起"多 Agent"突然爆火，它以前就存在吗？② 从学术概念到工程落地，这条线上的关键节点有哪些？③ 为什么协议化（MCP/A2A）是这个领域的分水岭？④ 站在 2026 年 9 月，多 Agent 的生态格局到底是什么样？

1. 多 Agent 不是新东西，LLM 让它"活"了

【一句话人话】多 Agent 系统就是"让多个各自有分工的智能体互相配合完成任务"——想法上世纪就有了，只是以前的"智能体"要么是死规则的专家系统，要么是学不出通用能力的模型，直到 LLM 出现。

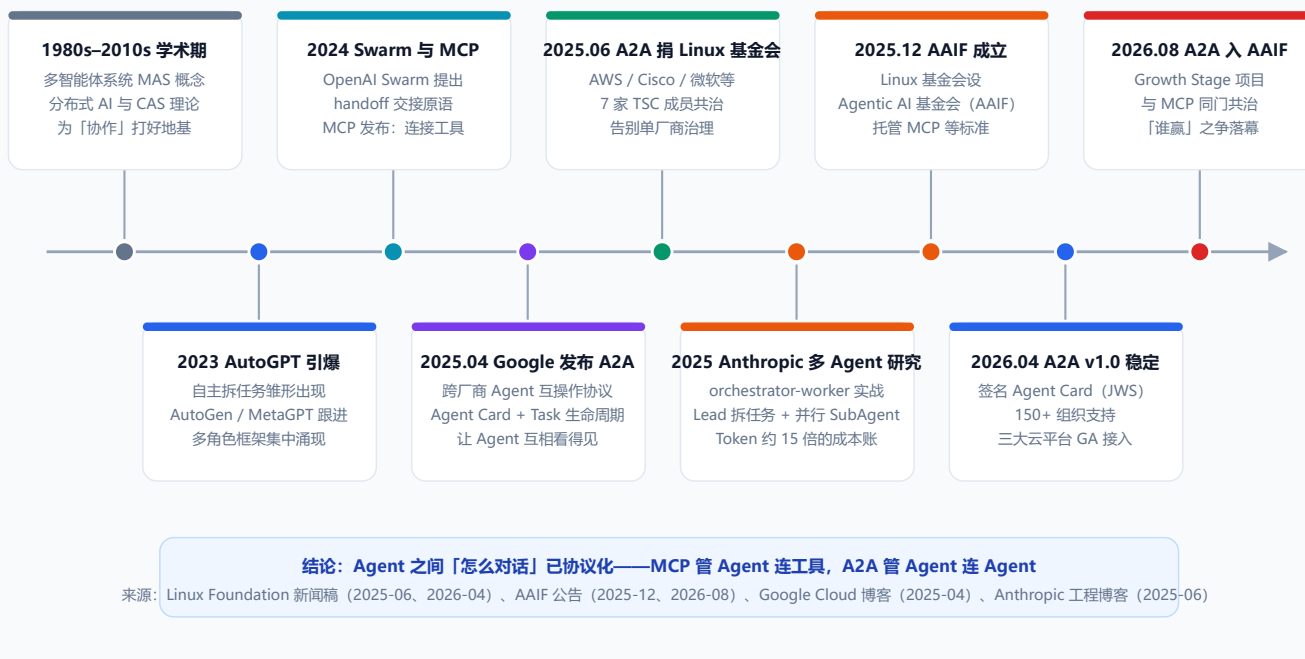
【生活类比】20 年前就有"连锁店管理模式"的理论（总部-区域-门店分工），但当时招不到能独当一面的店长；LLM 出现相当于突然有了一大批"能听懂话、会干活的通用员工"，管理理论立刻能落地了。

- **1980s–2010s**：分布式人工智能（DAI）与多智能体系统（MAS）是正经学术方向，研究合同网协议、博弈、协商机制；复杂适应系统（CAS）理论（圣塔菲研究所）提供了"简单个体涌现复杂行为"的思想底座。
- **那时的困境**：Agent 的"大脑"是规则引擎或窄模型，环境一变就崩。**缺的不是协作理论，是能理解模糊指令的通用大脑。**
- **LLM 改变一切**：GPT-4 级模型让"用一个自然语言 Prompt 定义一个能理解任务的员工"成为可能，MAS 四十年积累的协作理论（分工、协商、通信协议）突然全部可用。

2. 工程演进时间线：9 个关键节点

多 Agent 系统演进时间线（截至 2026-09）

从学术 MAS 到协议化协作：A2A 与 MCP 先后进入 Linux 基金会治理



优先级排序（先记这四个）：

- 2023, Autonomous Agent 元年**：AutoGPT/BabyAGI 展示"LLM 自己拆任务自己跑"的雏形——虽然脆弱失控，但证明了方向；AutoGen（微软）与 MetaGPT 随后把"多角色对话协作"工程化。
- 2025.04, Google 发布 A2A 协议**：Agent 之间怎么互相发现、怎么委托任务的开放标准。
- 2025.06, A2A 捐赠 Linux 基金会**：AWS、Cisco、Google、Microsoft、Salesforce、SAP、ServiceNow 七家组成技术指导委员会（TSC）共治（来源：Linux Foundation 新闻稿，2025-06-23）。
- 2026.08, A2A 进入 AAIF**：8 月 17 日被 Linux 基金会旗下 Agentic AI Foundation (AAIF) 接纳为 Growth Stage 项目，与 MCP 同属一个中立治理屋檐下（来源：AAIF 公告，2026-08-17）。

其余节点：**2024** OpenAI 发布 Swarm（实验性 handoff 原语）、Anthropic 发布 MCP（11 月）；**2025 年中** Anthropic 发表《How we built our multi-agent research system》工程复盘；**2025.12** AAIF 成立（创始项目：MCP、Block 的 goose、OpenAI 的 AGENTS.md，由 Anthropic/Block/OpenAI 联合创办）；**2026.04** A2A v1.0 稳定版发布（签名 Agent Card，宣称 150+ 支持组织）。

3. 协议化是分水岭：从"私下勾兑"到"公开标准"

【一句话人话】协议化之前，Agent 之间的协作全靠各家框架自定义的私有接口——像每个部门一套暗号，跨部门就抓瞎；协议化之后，协作有了"普通话"。

阶段	协作方式	问题
前协议期（2023–2025 上半年）	框架内私有对象传递（AutoGen 的 message、CrewAI 的 task 内部结构）	换框架即重写；跨厂商不可达
协议期（2025 下半年起）	MCP（连工具）+ A2A（连 Agent）开放标准	尚在普及，生态碎片在收敛

对 Java 开发者的直观理解：**MCP 之于工具，类似 JDBC 之于数据库；A2A 之于 Agent，类似 RPC/REST 之于微服务**——你的订单服务不需要知道库存服务内部用什么技术栈，看服务契约（Agent Card ≈ 服务注册与发现）就能调用。

4. 2026 年 9 月的生态快照

- **治理格局**：MCP 与 A2A 已同属 AAIF（Linux 基金会），"两个协议谁赢"的争论基本落幕——官方口径是**互补**：MCP 管 Agent→工具，A2A 管 Agent→Agent（来源：AAIF 2026-08 公告）。
- **平台落地**：Microsoft Copilot Studio、Azure AI Foundry、Amazon Bedrock AgentCore、Google Vertex AI Agent Builder 均已 GA 支持 A2A（来源：Linux Foundation 2026-04 周年公告）。
- **框架格局**：LangGraph（图编排/企业可控）、OpenAI Agents SDK（轻量 handoffs）、CrewAI（角色化快速起步）、Microsoft Agent Framework（Azure 企业向）为主流；有技术社区报道称 AutoGen 转入维护模式、建议新项目改用 MAF/AG2（来源：app-lab.ai 2026 综述，非官方口径，**截至 2026-09 未见微软官方文档明确宣布 AutoGen EOL**）。
- **仍未定论**：AAIF 成员数各来源口径不一（49 家创始 → 250+ 的增长被多家媒体引用但非官方实时数字）；A2A 生产环境采用率的权威统计**截至 2026-09 未见权威披露**。

【本节自测】

1. **多 Agent 想法早就有，为什么 2023 年才爆发？** 要点：缺"能理解模糊指令的通用大脑"；LLM 填上了大脑，MAS 四十年协作理论立刻可用。
2. **A2A 是谁、什么时候、以什么方式进入 Linux 基金会的？** 要点：Google 2025-04 发布，2025-06-23 捐赠 Linux 基金会，七家 TSC 共治；2026-08-17 成为 AAIF Growth Stage 项目。
3. **为什么说"谁赢"之争已落幕？** 要点：MCP 与 A2A 分属两层（连工具 vs 连 Agent），互补不竞争，且已被同一基金会治理。
4. **AAIF 是什么、谁创办的？** 要点：Linux 基金会 2025-12 宣布成立的 Agentic AI Foundation；Anthropic、Block、OpenAI 联合创办；创始项目 MCP/goose/AGENTS.md。
5. **A2A v1.0 带来了什么关键增强？** 要点：签名 Agent Card（JWS 验证身份）、多租户、版本协商（2026-04 稳定）。
6. **给 Java 开发者类比 MCP 和 A2A。** 要点：MCP ≈ JDBC（连外部能力），A2A ≈ 微服务间 RPC（Agent 互调），Agent Card ≈ 服务注册与发现。
7. **本章时间线的四条"必记"节点是什么？** 要点：2023 自主 Agent 元年；2025.04 A2A 发布；2025.06 捐 Linux 基金会；2026.08 入 AAIF 与 MCP 同门。

007 • 技术篇导航 (techniques/)

本章 8 节按"问题 → 概念 → 原理 → 进阶 → 实战 → 生产 → 对比 → 自测"递进，建议顺序阅读；每节开头有【本节问题】，结尾有【本节自测】。

节	回答的问题	配图
01-问题篇	单 Agent 的天花板在哪？什么时候 不该 拆？	007-02
02-概念篇	多 Agent 的 11 个核心术语到底是什么？	007-03
03-原理篇	Agent 之间到底传什么、怎么传、为什么隔离？	—
04-进阶篇	四种拓扑怎么选？A2A 协议怎么工作？	007-04 / 007-05
05-实战篇	不用框架，Python 手写点价评审小组？	007-06
06-生产篇	生产上会怎么翻车？怎么防？	007-07
07-对比篇	框架怎么选？什么时候坚持单 Agent？	—
08-自测篇	概念题 + 场景题 + 面试题自测	—

贯穿案例：点价评审小组（主管 + 抽取员/风控员/定价员），出自中基集团 CTRM 贸易系统的点价评审流程；读者默认画像为 Java 11 + Vue 2.7 + MySQL 的 CTRM 开发者，所有类比优先映射到 Java 生态。

先修：003-上下文工程（context rot）、004-单agent设计模式（ReAct/工具循环）、006-MCP协议。

01 · 问题篇：单 Agent 什么时候不够用

【本节问题】① 单 Agent 的能力上限由什么决定，具体卡在哪四件事上？② 怎么判断我的场景"真的需要"多 Agent？③ 为什么 Anthropic 说"能单 Agent 就单 Agent"，多 Agent 的代价到底是什么？④ 点价评审这个场景，拆与不拆的账分别怎么算？

1. 先承认：单 Agent 的天花板是结构性的

【一句话人话】单 Agent 的天花板不靠换更强的模型解决——上下文窗口、工具列表、串行执行、单一 Prompt 这四件事是**结构性**限制，模型再强也只是把墙推远一点。

单 Agent 的天花板与「该不该拆」的信号

先承认单 Agent 的极限，再对照清单决定要不要上多 Agent

单 Agent 的四大天花板

① 上下文装不下

长任务的中间结果、检索资料、工具返回全部堆进一个窗口，早期信息被稀释

CTRM 类比：评审要查 20 份单据，一屏放不下

② 工具太多，选择混乱

工具超过十几个，模型选错率明显上升

描述互相打架，参数更容易填错

拆开每个 Agent 只带 3~5 个专属工具

③ 串行慢，一条链跑到底

步骤互相等待：先查库、再算价、再写报告

总耗时 = 各步之和，无法并行提速

多 Agent 可 `asyncio.gather` 并行分头干

④ 职责混杂，Prompt 膨胀

抽取 + 风控 + 定价塞进同一个 Prompt

规则互相干扰，容易互相矛盾

一个 Agent

该不该拆？对照两张信号清单

✓ 拆分信号（满足 2 条以上再拆）

- 任务可分解为边界清晰的独立子任务
- 子任务互相独立，能并行省时间
- 不同子任务需要不同的工具或专长
- 单 Agent 的上下文确实装不下了
- 需要多视角交叉验证（如风控 + 定价）

点价评审：抽取/风控/定价三类子任务独立、可并行 → 适合拆

X 不该拆的信号（Anthropic 观点）

- 能单 Agent 就单 Agent——多 Agent 是双刃剑
- 多 Agent Token 消耗约为单 Agent 的 15 倍
- 子任务强耦合、要共享大量上下文
- 一次问答能解决的事（拆了纯属浪费）
- 需要严格确定性：直接写代码而非堆 Agent

来源：Anthropic《How we built our multi-agent research system》(2025-06)

裁定顺序：先优化 Prompt 与工具 → 不行再试「工作流固化」→ 确需并行与专长分工才上多 Agent

拆的收益 = 并行提速 + 上下文隔离；拆的代价 = 15 倍 Token + 协调复杂度。赚不赚，先算账再动工

天花板 ①：上下文装不下

长任务的中间结果、检索到的资料、每次工具返回，全部堆进同一个窗口。评审一笔点价单要查合同、授信、行情、历史成交，资料塞满后早期信息被稀释——这就是 003 讲的 **context rot**：上下文越长，注意力越稀，前面读的"客户授信 500 万"到第 50 个工具调用时已经"记不清"了。

天花板 ②：工具太多，选择混乱

工具超过十几个，模型选错率明显上升：描述互相打架（`query_price` 和 `get_market_price` 区别是什么？）、参数更容易填错。研究口径常见经验值：一个 Agent 的工具在个位数到 15 个以内表现最稳。多 Agent 拆分后，每个 Worker 只带 3~5 个专属工具。

- 10 / 38 -

天花板 ③：串行慢，一条链跑到底

单 Agent 内部步骤天然串行：查合同 → 等结果 → 查授信 → 等结果 → 查行情.....总耗时 = 各步之和。而這些查詢互相獨立，本可并行。多 Agent 把獨立查詢分給三個 Worker 同時跑，牆鐘時間接近"最慢那個"而不是"全部之和"。

天花板 ④：职责混杂，Prompt 膨胀

把"抽取规则 + 风控规则 + 定价规则"塞进一个系统提示：规则互相干扰（抽取要求逐字引用原文，定价要求大胆推断，风格拧着来）、角色没有归属感、改一处规则影响全部。这就是 Java 老兵都懂的道理：**一个类 3000 行、40 个方法的 God Class，怎么改都是雷。**

2. 拆分信号清单：满足 2 条以上再拆

信号	判断问题	点价评审对照
任务可分解	能划出边界清晰的独立子任务吗？	抽取/风控/定价三类子任务边界清晰 ✓
可并行	子任务互相独立、能同时跑吗？	三个查询互不依赖 ✓
需要不同专长/工具	各子任务的工具集和 Prompt 风格差异大吗？	抽取要"逐字引用"，定价要"行情比对" ✓
上下文确实装不下	单窗口塞下全部资料后表现明显劣化吗？	20+ 份单据塞不下 ✓
需要交叉验证	结论需要多视角互相校验吗？	风险与定价互相印证 ✓

3. 不该拆的信号：Anthropic 的冷水要接住

Anthropic 在《How we built our multi-agent research system》（2025-06）里明确给出两句话：

"When to use multi-agent": **能单 Agent 就单 Agent**；多 Agent 是双刃剑。Token 账：**多 Agent 系统的 Token 消耗约为单 Agent 聊天的 15 倍**（Agent 模式约为聊天的 4 倍，多 Agent 放大到约 15 倍）。

不该拆的信号：

- **一次问答能解决的事**：拆出编排器纯属浪费；
- **子任务强耦合**：B 需要 A 的完整推理过程才能干活——隔离上下文反而制造信息断层；
- **追求确定性**：固定规则流程（比如金额换算、日期计算）应该写代码，不该堆 Agent；
- **Token 预算紧张**：15 倍不是修辞，是按量计费的账单。

4. 点价评审的拆账

拆的收益：抽取/风控/定价并行（3 个 Worker 各 40s，总耗时 ≈ 40s 而非 120s）；上下文隔离（风控员只看授信与合同，不被行情数据干扰）；角色聚焦（每份 Prompt 短而专）。

拆的代价：Token 约为单 Agent 的 15 倍量级；主管要做任务拆解与结果校验（这本身也花 Token）；多了一层协调失败的可能（06-生产篇的主题）。

结论：评审是高频核心业务、错误代价高、子任务天然独立——**值得拆**。反过来，“帮我查一下这个客户的授信余额”这种一句话任务，直接单 Agent + 一个工具就好。

优先级要点：1. **先优化单 Agent**（Prompt、工具描述、上下文压缩），不行再考虑拆；2. **再试 workflow 固化**（把确定步骤写死成代码流水线），只有真正需要“多视角判断 + 并行”才上多 Agent；3. **拆之前先算 Token 账：**预估收益（时间/质量）能否覆盖 15 倍成本。

【本节自测】

1. **单 Agent 的四大天花板是什么？** 要点：上下文装不下、工具多选择乱、串行慢、职责混杂 Prompt 膨胀；都是结构性限制。
2. **context rot 和天花板①什么关系？** 要点：上下文越长注意力越稀释（003），多 Agent 用“上下文隔离”对症。
3. **拆分信号清单的五条是什么？** 要点：可分解、可并行、需不同专长/工具、上下文装不下、需交叉验证；满足 2 条以上再拆。
4. **Anthropic 给的 Token 倍数是多少？怎么算的？** 要点：多 Agent $\approx$ 单 Agent 聊天的 15 倍；Agent 模式 $\approx$ 4 倍，多 Agent 协调进一步放大（Anthropic 工程博客 2025-06）。
5. **为什么“子任务强耦合”不该拆？** 要点：B 需要 A 的完整推理链，隔离上下文制造信息断层，协调成本 $>$ 收益。
6. **点价评审为什么值得拆？** 要点：高频核心、错误代价高、三个子任务独立可并行、需要交叉验证。
7. **上多 Agent 前的正确动作顺序？** 要点：优化单 Agent $\rightarrow$ 工作流固化 $\rightarrow$ 算 Token 账 $\rightarrow$ 才是多 Agent。

02 · 概念篇：核心概念精讲（11 张概念卡）

【本节问题】① 多 Agent 领域的术语为什么这么多，哪些是本质哪些是包装？② "角色、编排者、通信、拓扑、handoff"这几个词到底精确指什么？③ 共享状态和私有上下文为什么要分开设计？④ A2A 协议里 Agent Card、Task、Artifact 各扮演什么角色？



概念卡 1：多 Agent 动机

【一句话人话】多 Agent 的动机只有一个：**单 Agent 的结构天花板（装不下、选不清、跑不快、管不住）真的卡住了业务。**【生活类比】不是招更多人的问题，是"一件事一个人干不完"才开岗位；一个人加加班就能干的活，开会协调反而更慢。- 坑：把多 Agent 当时髦，是 2026 年最常见的架构事故源头。

概念卡 2：角色 (Role / Persona)

【一句话人话】角色 = 给一个 Agent 的**人设 + 职责边界 + 输出要求**，通常写进系统提示。【生活类比】岗位 JD：你是风控专员，你只管风险，产出物格式是这份模板。- 要素：身份（你是谁）+ 边界（只做什么、明确不做什么）+ 交付（输出 Schema）。- 坑：角色只写"你是资深风控专家"没用——没有边界和交付物定义的角色是装饰品。

概念卡 3：编排者 (Orchestrator / Supervisor)

【一句话人话】编排者是**拆任务、派活、收结果、做仲裁**的总调度 Agent，其他 Worker 只对它负责。【生活类比】项目主管：把需求拆成工单分给专员，收齐结果汇总向上汇报，专员之间不直接扯皮。 - 也叫 Supervisor / Lead Agent / 主管 Agent；Anthropic 的 orchestrator-worker 模式主角。 - 坑：编排者自己也是 LLM，拆解也会错——它的输出同样需要校验。

概念卡 4：通信 (消息 / 共享黑板)

【一句话人话】Agent 间通信两种基本形态：**点对点消息** (A 直接发内容给 B) 和**共享黑板** (大家读写同一块公共空间)。【生活类比】发微信 (消息) vs 会议室白板贴便签 (黑板)。 - 消息：责任清晰、易追踪，适合任务委派；黑板：松耦合、异步，适合多方共享事实 (如"今天基准价")。 - 坑：黑板写入无 Schema 约束时，会退化成"谁都能涂鸦的白板"，信息互相污染。

概念卡 5：拓扑 (Topology)

【一句话人话】拓扑决定**谁跟谁能说话、信息往哪流**——星型、流水线、层级、网状四种基本形。【生活类比】组织沟通结构：全部汇报给主管 (星型)、流水线一道工序接一道 (流水线)、逐级汇报 (层级)、圆桌自由讨论 (网状)。 - 拓扑是架构决策，不是实现细节：它直接决定成本、延迟、可观测性与失败模式。 - 详见 04-进阶篇与 007-04 对比图。

概念卡 6：Handoff 交接

【一句话人话】Handoff = A 把任务**连同上下文一起转交**给 B，B 从此接管，A 退出。【生活类比】工单转派：不是甩一句"你处理下"，而是连材料、背景、验收标准一起打包移交。 - OpenAI Swarm/Agents SDK 把它做成一等原语：转交给谁 = 切换到谁的 Prompt 与工具集。 - 坑：只转任务不转上下文 = 甩锅，是失败模式②的根源。

概念卡 7：共享状态 vs 私有上下文

【一句话人话】共享状态是**大家都要对齐的事实** (公共白板)；私有上下文是**各自工作台上的草稿** (不该外泄也不该塞给别人)。【生活类比】项目群公告 (共享) vs 你自己的草稿纸 (私有)。 - 多 Agent 的核心收益就是私有上下文隔离：风控员不必看到行情 API 的原始返回。 - 坑：全共享 → 互相污染、窗口爆炸；全私有 → 口径不一致 ("数量"vs"手数")。按事实类型分层共享。

概念卡 8：Agent 团队 (Agent Team / Crew / Group Chat)

【一句话人话】Agent 团队 = 一组角色化 Agent + 一套协作规则的**统称**，各框架叫法不同本质相似。【生活类比】项目组：CrewAI 叫 crew，AutoGen 叫 group chat，LangGraph 里是一张图——都是"人 + 协作制度"。 - 别被名词迷惑：评估任何框架的"团队"只需看三件事——怎么定角色、怎么定通信、怎么定终止条件。

概念卡 9：并行 Sub-Agent

【一句话人话】主 Agent 拆出多个子任务后**同时派发**给多个 SubAgent 并行执行，收回来的是摘要/交付物而非全程日志。 【生活类比】老板群发三个任务给三个专员，收三份报告，不旁听每个人的工作过程。 - Anthropic 实测：并行工具调用为多 Agent 系统节省约 90% 的时间（来源：Anthropic 工程博客 2025-06）。 - 工程实现即 `asyncio.gather` / `CompletableFuture.allOf`，05-实战篇落地。

概念卡 10：A2A 协议 (Agent2Agent)

【一句话人话】A2A 是**跨厂商 Agent 互操作开放协议**：用 Agent Card 自我介绍，用 Task 承载任务生命周期，用 Artifact 交付产物。 【生活类比】企业间合作标准：先交换公司简介与资质（Agent Card），签合同定里程碑（Task），验收交付物（Artifact）。 - 三要素：**Agent Card**（`/well-known/agent-card.json` 自描述能力与鉴权方式，v1.0 起支持 JWS 签名）、**Task 状态机**（submitted → working → completed/failed，可 input-required 暂停等澄清）、**Artifact**（任务产物）。 - 现状：2025-04 Google 发布 → 2025-06 捐 Linux 基金会 → 2026-04 v1.0 → 2026-08 入 AAIF（来源：Linux Foundation / AAIF 公告）。 - 与 MCP 互补：MCP 连工具，A2A 连 Agent（详见 04-进阶篇）。

概念卡 11：成本放大 (Token × 15)

【一句话人话】多 Agent 的并行与隔离都有代价：**Token 消耗约为单 Agent 聊天的 15 倍**——每个 Worker 重复读背景、每次 handoff 携带内容、编排本身都要花钱。 【生活类比】开会本身不产出，但每个参会者的时间都在计费；人多不等于便宜，等于贵得快。 - Anthropic 口径：Agent ≈ 4× 聊天，多 Agent ≈ 15× 聊天（2025-06 工程博客）。 - 防御见 06-生产篇：预算护栏、模型分级、先抽样试跑。

概念速查表

概念	一句话	Java 心智映射
角色	人设 + 边界 + 交付	岗位 JD + DTO 输出约定
Orchestrator	拆解派活收结果的调度	工作流引擎 / 网关路由
通信	消息 or 黑板	RPC 消息 vs 共享缓存
拓扑	谁能和谁说话	服务依赖拓扑
Handoff	带上下文的任务转交	工单转派 + 材料移交
共享 vs 私有	公共事实 vs 工作草稿	共享缓存 vs 线程栈
Agent 团队	角色 + 协作规则的统称	项目组
并行 SubAgent	同时派发只收摘要	<code>CompletableFuture.allOf</code>
A2A	Agent 互操作协议	微服务 RPC 标准 + 服务发现
成本放大	$\text{Token} \approx 15 \times$	每次跨服务调用的序列化开销 $\times N$

【本节自测】

1. **角色的三要素是什么？** 要点：身份人设、职责边界（含明确不做什么）、输出交付（Schema）。
2. **点对点消息和共享黑板怎么选？** 要点：任务委派与审计要求高 → 消息；多方共享事实、松耦合 → 黑板；黑板必须配 Schema。
3. **Handoff 和"甩锅"的区别？** 要点：是否携带上下文四件套（任务、数据、Schema、边界）。
4. **多 Agent 的核心收益为什么是"私有上下文隔离"？** 要点：每个 SubAgent 只装自己需要的上下文，对抗 context rot。
5. **Agent Card 是什么、放在哪、v1.0 加了什么？** 要点：自描述 JSON，`/.well-known/agent-card.json`，v1.0 支持 JWS 签名验证身份。
6. **A2A 的 Task 状态机主链路？** 要点：submitted → working → completed/failed；可 input-required 暂停等待澄清。
7. **15 倍 Token 的来源有哪些？** 要点：Worker 重复读背景、handoff 载荷、编排/校验自身消耗、并行放大。
8. **Agent 团队各框架叫法？** 要点：CrewAI crew、AutoGen group chat、LangGraph 图编排、OpenAI Agents SDK handoffs 网络。

03 • 原理篇：协作模式与通信解剖

【本节问题】① Agent 之间一次协作的完整数据流长什么样？② 消息里到底该传什么——完整历史、摘要还是结构化产物？③ "怎么传"有哪几条通道，各适合什么场景？④ 上下文隔离为什么是多 Agent 的第一性收益，它和 context rot 什么关系？

1. 一次协作的解剖：五段数据流

以点价评审为例，一笔任务从进到出的完整数据流：



对 Java 开发者，这就是一次**扇出-扇入 (scatter-gather)**：`CompletableFuture.allOf` 派发、各服务返回 DTO、网关聚合校验。唯一区别是"服务"换成了带 Prompt 与工具循环的 Agent。

2. 传什么：三种载荷的取舍

载荷形态	内容	优点	缺点	适用
完整历史	把 A 的对话/工具日志全量给 B	信息零损失	Token 爆炸；把 B 拖进 context rot	强耦合接续（B 必须复现 A 的推理）
摘要	A 用自然语言压缩工作结果	便宜、聚焦	压缩丢细节，关键数字可能被"大概化"	松耦合咨询（B 只要结论）
结构化产物	约定 Schema 的 JSON 交付物	机器可校验、口径统一、可入库	前期要设计 Schema	生产系统默认选择

生产默认：结构化产物。点价评审中主管给风控员的不是"合同全文和历史"，而是抽取员产出的 `extraction.json` ——风控员需要的字段就在里面，一行不多。

优先级要点：1. 子任务间传**结构化产物**，别传聊天记录；2. 需要"再加工"的场景传**摘要**，但数字必须原文保留（禁止模型四舍五入金额）；3. **完整历史**只留给"接手未完成任务"（如 handoff 换人重试），且要配截断策略。

3. 怎么传：三条通道

通道	机制	类比	适用
进程内直接调用	函数/SDK 调用，共享内存对象	同 JVM 内方法调用	同一编排进程内的 Worker（05-实战篇走这条）
共享存储黑板	Redis/DB/文件作为公共空间	共享缓存表	异步协作、多方共享事实
A2A 网络调用	HTTP + Agent Card 发现 + Task 生命周期	微服务 RPC + 服务注册发现	跨进程、跨团队、跨厂商 Agent

经验法则（社区共识，Alice Labs 2026 实践综述）："A2A between crews, agents-as-tools inside a crew"——同一进程内的 Agent 别走 HTTP，把对方当工具直接调；跨边界（跨服务/跨组织）才上 A2A。

4. 上下文隔离：多 Agent 的第一性收益

【一句话人话】上下文隔离 = **每个 SubAgent 只带自己那部分上下文**：主管不发全文，只发该子任务需要的最小集合。【生活类比】给专员派活只给相关材料，而不是把整个档案室搬到他桌上。

为什么这是第一性收益：

- 1. **对抗 context rot** (003)：上下文越长，注意力越稀释。风控员的窗口里只有合同 + 授信数据，没有行情噪声，关键信息不被冲淡；
- 2. **省 Token**：不重复给每个 Worker 发全量资料；
- 3. **降干扰**：抽取的"逐字引用"与定价的"合理推断"两种 Prompt 风格不再互相污染；
- 4. **可并行**：隔离是并行的前提——不共享可变状态，才能放心 gather。

隔离的反面是口径漂移：三个 Worker 各自为政后，"数量"可能被抽成吨、被风控读成手数。所以隔离必须配**共享契约**——交付物 Schema 与公共事实字典（见 06-生产篇防御①）。

5. 协作模式的三个层次（由简到繁）

层次	形态	主管职责	适用
L1 固定委派	子任务预先写死，主管只派发与汇总	派活 + 校验	流程确定的评审/审核类
L2 动态拆解	主管 LLM 自行决定拆几个、拆成啥	规划 + 编排	开放任务（研究、巡检）
L3 自主协作	Worker 间可直接通信、互派任务	定规则 + 仲裁	对等辩论、复杂协商

优先级要点：**生产从 L1 起步**——L2 已经引入"拆解错误"风险，L3 的协调成本 ($O(n^2)$ 通信) 与死循环风险只值得在确需"辩论"的场景支付。点价评审用 L1：拆解逻辑写死在代码里，主管 LLM 只负责汇总校验，这是 05-实战篇的取向。

【本节自测】

1. **一次多 Agent 协作的五段数据流？** 要点：接收 → 拆解派发 → 并行执行 → 回传产物 → 汇总校验交付 (scatter-gather) 。
2. **三种载荷怎么选？** 要点：默认结构化产物；松耦合咨询用摘要（数字原文保留）；完整历史仅用于接手重试且配截断。
3. **三条通道各对应什么 Java 类比？** 要点：进程内调用 $\approx$ 同 JVM 方法调用；共享黑板 $\approx$ 共享缓存；A2A $\approx$ RPC + 服务发现。
4. **"A2A between crews, agents-as-tools inside a crew"什么意思？** 要点：进程内直接调，跨边界才走协议，别为同进程 Agent 加 HTTP 跳数。
5. **上下文隔离的四个收益？** 要点：对抗 context rot、省 Token、降风格干扰、可并行。
6. **隔离带来的新风险是什么、怎么补？** 要点：口径漂移；用共享交付物 Schema + 公共事实字典对齐。
7. **三个协作层次，生产该从哪层起步？** 要点：L1 固定委派起步；L2/L3 各自的风险与适用。

04 · 进阶篇：编排拓扑与 A2A 协议

【本节问题】① 四种拓扑各自的优劣和适用边界是什么？② A2A 协议的 Agent Card、Task 状态机、Artifact 具体怎么工作？③ A2A 和 MCP 到底怎么分工、怎么配合？④ Anthropic 的多 Agent 研究系统沉淀了哪些可直接抄的工程经验？

1. 四种拓扑对比

四种协作拓扑对比：星型 · 流水线 · 层级 · 网状

拓扑决定「谁跟谁能说话」——也决定了瓶颈、成本与可观测性

星型（中心化编排 · Orchestrator-Worker）

优：拆解派活汇总一处管，可控、易观测、好审计
缺：中心是瓶颈也是单点，挂了全停
适用：任务可拆解、结果要汇总——点价评审小组
类比：项目主管带三个专员，活分下去、报告交上来
首选默认型：90% 的业务多 Agent 场景从它开始

流水线（Pipeline · 串行接力）

上一站的输出 = 下一站的输入（结构化传递）
优：职责单一可测试，节点可插拔、可回放
缺：吞吐卡在最慢节点；前段出错全链污染
适用：固定 SOP——抽取→清洗→审核→报告
类比：工厂流水线，一人一道工序，不越位
最稳型：流程确定时优先固化成流水线

层级（树形 · 分治 Hierarchical）

优：分治可扩展，超大任务层层拆到可执行
缺：信息层层衰减，延迟与成本逐级叠加
适用：超大广度任务——全库盘点、全站巡检
类比：公司组织架构，逐级汇报、逐级拆解
规模型：任务大到一层拆不动时才升级到它

网状（去中心 · 对等 Mesh）

优：无单点有弹性，对等协作可辩论、可互审
缺：两两通信 $O(n^2)$ ，成本高、难观测、易死循环
适用：对等讨论、交叉验证、群聊式决策
类比：圆桌会议人人可发言——热闹但容易跑题
慎用型：除非真需要「辩论」，否则别选

选型口诀：能流水线就流水线，要汇总用星型，大到一层拆不动用层级，真要辩论才上网状

拓扑	结构	优点	缺点	适用	点价评审对照
星型	主管 + N Worker，放射通信	可控、易观测、好审计；上下文在主管集中	主管是瓶颈与单点	可拆解 + 需汇总	✓ 正是这个
流水线	A→B→C→D 线性接力	职责单一可测试；节点可插拔	吞吐卡最慢节点；前段错全链污染	固定 SOP	抽取→清洗→审核→归档
层级	树状，逐级拆解	分治可扩展，超大任务拆到可执行	信息层层衰减；延迟成本逐级叠加	超大广度任务	全量历史单据盘点
网状	对等全连	无单点有弹性；可辩论互审	$O(n^2)$ 通信；难观测；易死循环	对等讨论/交叉验证	风险争议时的复核辩论

选型口诀：能流水线就流水线，要汇总用星型，大到一层拆不动用层级，真要辩论才上网状。

工程细节： - 星型主管负载 = N 份子任务规划 + N 份结果校验。Worker 数量建议 3~5 个：再多主管上下文又开始 rot——**星型的天花板是主管的上下文**； - 层级 = 星型的递归嵌套（每个中层又是一个小主管），代价是每层都有信息损耗，能不嵌就不嵌； - 网状必须配全局轮次上限与终止仲裁者，否则乒乓循环烧钱（06-生产篇失败模式③）。

2. A2A 协议详解

【一句话人话】A2A 给"Agent 调用另一个 Agent"定了标准：先看名片（Agent Card）确认能力和鉴权，下单立任务（Task），看状态机跟踪进度，验收交付物（Artifact）。

2.1 Agent Card：自我介绍

部署在 `https://{host}/.well-known/agent-card.json` 的 JSON 文档，内容包括：能力描述（支持什么技能）、服务地址、鉴权方案（APIKey/HTTP Bearer/OAuth2/OIDC/mTLS 等，v1.0 支持 **JWS 签名卡**验证身份）。调用方先拉名片决定"能不能调、怎么调"。类比：微服务的服务注册信息 + OpenAPI 文档合体。协议只**声明**鉴权方案，不代执行——加不加固是实现者的责任。

2.2 Task：带状态机的工作单元

状态	含义
submitted	已受理
working	执行中（可多次更新进度）
input-required	暂停，等调用方补充澄清
completed	完成，附 Artifact
failed	失败，附错误
canceled	取消

关键设计：Agent 对调用方是**不透明**的——只能看到 Task 状态与消息，看不到对方内部推理链。这就是"把 Agent 当工具调"与"把 Agent 当对方委托"的本质区别：前者是一次函数调用，后者是带生命周期的委托合同。传输绑定：JSON-RPC 2.0、gRPC、HTTP+JSON/REST；流式进度用 SSE（`sendMessageStream`）。

2.3 与 MCP 的关系：互补两层

A2A 与 MCP：同一套 Agent 栈的互补两层 (2026-09)

MCP 管纵向「Agent 连工具」，A2A 管横向「Agent 连 Agent」——不是二选一，是一起用



维度	A2A	MCP
连接对象	Agent ↔ Agent (对等协作)	Agent → 工具 / 数据源
工作单元	Task (状态机 + 多轮消息 + Artifact)	一次工具调用 / 资源读取
治理现状	AAIF Growth Stage (2026-08 接纳)	AAIF 创始项目 (2025-12 起托管)

生产栈通行做法: MCP 给每个 Agent 发工具, A2A 让 Agent 跨界协作; 同进程内的 Agent 别走 HTTP

维度	A2A	MCP
连接对象	Agent ↔ Agent (对等, 跨组织)	Agent → 工具/数据
工作单元	Task (状态机 + 多轮消息 + Artifact)	一次工具调用/资源读取
发现机制	Agent Card (well-known URL)	initialize 能力协商
对调用方暴露	不透明 (只有状态和消息)	显式 (工具 schema、资源内容)
治理 (2026-09)	AAIF Growth Stage (2026-08 接纳)	AAIF 创始项目 (2025-12 起托管)

(来源: AAIF 2026-08 公告、Linux Foundation 2025-06 新闻稿。)

灰区提示: MCP 也在加实验性 **tasks 原语** 支持长任务, 但 MCP Server 本质仍是“能力提供方”而非“自主协商方”; 反过来, 同一个 Agent 可以同时用两种方式暴露——给同步调用方当 MCP 工具, 给编排方当 A2A 远程 Agent。选型判断: 对方有自己的推理循环、可能中途提问、可能跑十分钟 → A2A; 固定输入输出的能力 → MCP 工具。

3. Anthropic 多 Agent 研究系统：可抄的工程经验

Anthropic 《How we built our multi-agent research system》(2025-06) 是公开资料里最完整的多 Agent 生产复盘, 四条可直接抄:

- Orchestrator-Worker 是主结构**: Lead Agent 负责拆解与委派, SubAgent 并行执行、只回摘要。研究类任务收益最大 (内部评估比单 Agent 聊天提升 90.2%), 因为子任务天然独立可并行;

2. **并行换时间**：并行工具调用与并行 SubAgent 共同节省约 90% 时间——但**仅当子任务独立时成立**；
3. **任务描述决定下限**：Lead 给 SubAgent 的任务必须写清目标、输出格式、**工具指引**（用哪些工具、别用哪些）与**努力程度**（effort pacing：该深挖还是浅尝）；SubAgent 不是“聪明到不用指令”的员工；
4. **15 倍 Token 是账不是梗**：Agent $\approx 4\times$ 聊天，多 Agent $\approx 15\times$ 聊天。只在任务价值足够高（用户付费研究、高风险评审）时启用；同时要处理失败模式：SubAgent 卡死、工具超时、上下文溢出——全部要有重试与熔断。

优先级要点：1. 抄结构：Lead 拆任务 + 并行 SubAgent + 只收摘要；2. 抄纪律：任务描述四件套（目标/格式/工具指引/努力程度）；3. 抄账本：上线前用 $15\times$ 口径估成本，按价值分级启用。

【本节自测】

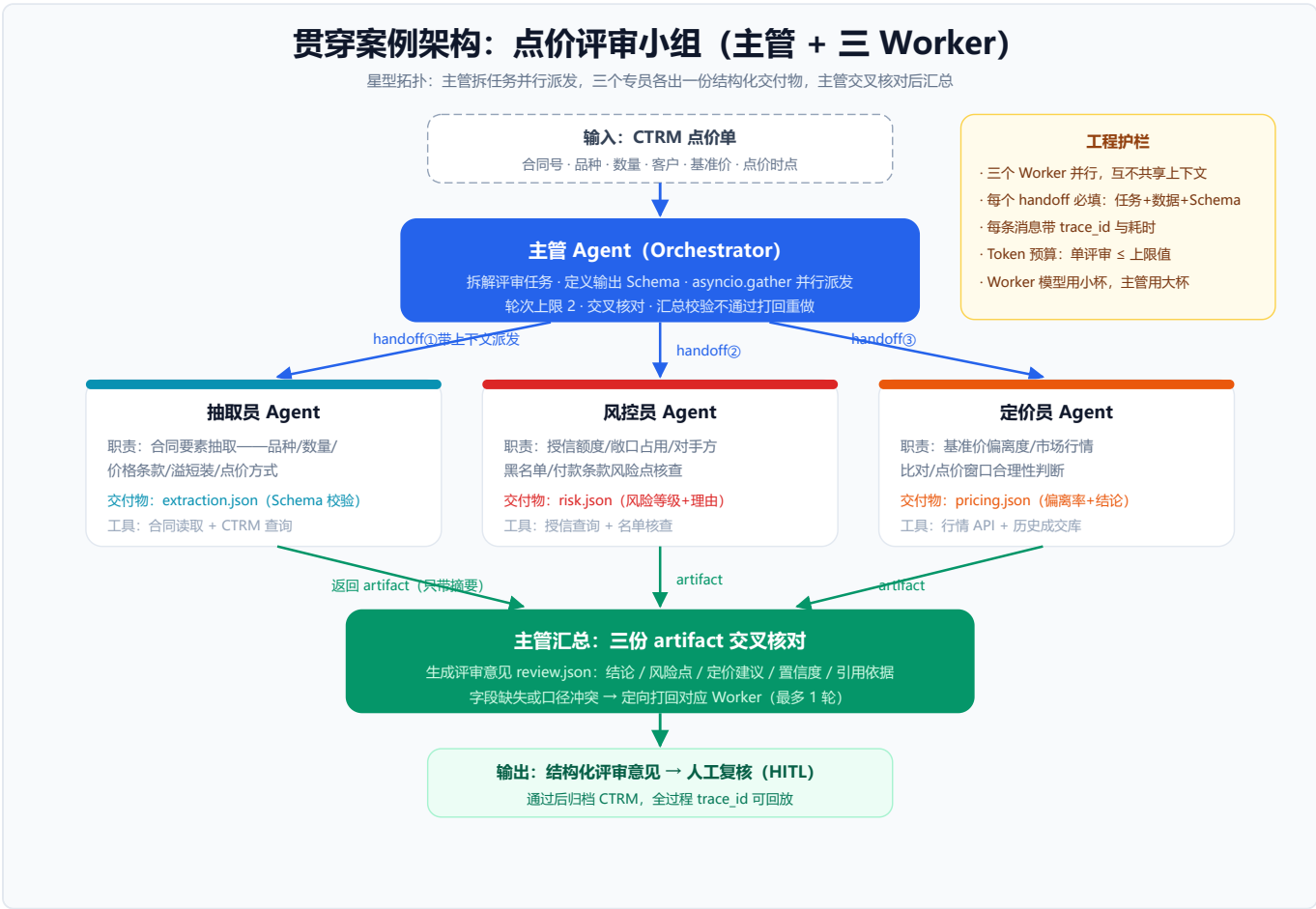
1. **四种拓扑各一句话优劣**？要点：星型可控但中心瓶颈；流水线稳定但卡最慢节点；层级可扩展但层层衰减；网状弹性但 $O(n^2)$ 且易死循环。
2. **星型拓扑的天花板在哪、Worker 数量建议**？要点：主管上下文；3~5 个。
3. **Agent Card 是什么、URL 约定、v1.0 增强**？要点：`.well-known/agent-card.json` 自描述；JWS 签名验证身份；协议只声明鉴权不代执行。
4. **Task 状态机全链路**？要点：submitted $\rightarrow$ working $\rightarrow$ (input-required) $\rightarrow$ completed/failed/canceled。
5. **A2A 与 MCP 的三层区别（对象/单元/暴露）**？要点：连 Agent vs 连工具；Task vs 单次调用；不透明 vs 显式 schema。
6. **什么时候把 Agent 暴露为 MCP 工具、什么时候用 A2A**？要点：固定输入输出 $\rightarrow$ MCP；有自主推理、需多轮澄清、长任务 $\rightarrow$ A2A。
7. **Anthropic 经验中“任务描述四件套”**？要点：目标、输出格式、工具指引、努力程度（effort pacing）。
8. **90.2% 与 $15\times$ 各指什么**？要点：内部研究评估相对单 Agent 聊天的性能提升；多 Agent 相对聊天的 Token 消耗倍数。

05 · 实战篇：手写一个多 Agent 协作（Python 全代码）

【本节问题】① 不用任何框架，多 Agent 协作的最小实现长什么样？② handoff 消息在代码里长什么结构？③ 并行派发与汇总校验怎么写？④ 这套手写代码和 LangGraph/OpenAI Agents SDK 的对应关系是什么？

0. 目标与运行方式

贯穿案例：点价评审小组——主管 Agent 拆解评审任务 → 并行派给三个 Worker（抽取员/风控员/定价员）→ 收齐三份结构化交付物 → 汇总生成评审意见。拓扑即 007-06 架构图：



- 仅依赖 `openai` 官方 SDK + 标准库 (`asyncio`)，无框架；
- 标题里的"双 Agent"是最小形态：把 `WORKERS` 砍到 1 个就是"主管 + 1 Worker"双 Agent 版；本篇按贯穿案例给完整的 1 + 3 版；
- Worker 模型用便宜档（生产即"模型分级路由"），主管用强档。

```
pip install openai>=1.40
export OPENAI_API_KEY=sk-xxxx # Windows PowerShell: $env:OPENAI_API_KEY="sk-xxxx"
python 007_multi_agent_review.py
```

1. 全部代码（单文件可跑）

```
# -*- coding: utf-8 -*-
"""
007_multi_agent_review.py — 点价评审小组（主管 + 三 Worker）
拓扑：星型（L1 固定委派）。主管不拆 LLM 任务（拆解逻辑写死），只做派发与汇总校验。
护栏：交付物 Schema 校验、轮次上限、trace_id 全链路、Token 预算口径。
"""

import asyncio
import json
import uuid
from dataclasses import dataclass, field, asdict
from openai import AsyncOpenAI

client = AsyncOpenAI()
MODEL_WORKER = "gpt-4o-mini" # 小杯模型：Worker 用（成本分级）
MODEL_LEAD = "gpt-4o" # 大杯模型：主管汇总用
MAX_TURNS = 6 # 每个 Worker 的工具轮次上限（防循环）

# ----- 1. 模拟 CTRM 工具（生产中替换为 MCP 工具 / DB 查询） -----

def get_contract(contract_no: str) -> dict:
    """查合同要素（模拟 CTRM 合同表）"""
    return {"contract_no": contract_no, "variety": "电解铜", "quantity_ton": 500,
            "price_clause": "M+1 月均价 + 300", "basis_price": 73850,
            "pricing_window": "2026-09-01 ~ 2026-09-30", "tolerance": "溢短装 ±2%"}

def get_credit(customer: str) -> dict:
    """查客户授信（模拟授信表）"""
    return {"customer": customer, "credit_limit": 5_000_000, "used": 4_620_000,
            "payment_terms": "货到 30 天", "blacklisted": False}

def get_market_price(variety: str, month: str) -> dict:
    """查行情（模拟行情 API）"""
    return {"variety": variety, "settle_month": month, "spot_avg": 74120,
            "prev_month_avg": 72900, "volatility": "中"}

# ----- 2. handoff 消息结构：四件套必填 -----

@dataclass
class Handoff:
    trace_id: str # 全链路追踪 ID
    from_role: str # 派发方
    to_role: str # 接收方
    task: str # 任务目标（做什么）
    input_data: dict # 输入数据（材料，而不是"你自己查"）
    output_schema: dict # 输出契约（验收标准）
    boundary: str # 边界（明确别做什么）

    def render(self) -> str:
        """把 handoff 渲染成发给 Worker 的消息正文"""
        return (f"[handoff {self.trace_id}] {self.from_role} → {self.to_role}\n"
                f"任务: {self.task}\n输入数据: {json.dumps(self.input_data, ensure_ascii=False)}\n"
                f"输出契约: {json.dumps(self.output_schema, ensure_ascii=False)}\n"
                f"边界: {self.boundary}")
```

```
# ----- 3. Worker 定义: 角色提示词 + 专属工具 + 输出契约 -----
```

```
EXTRACTION_SCHEMA = {"contract_no": "str", "variety": "str", "quantity_ton": "number",
                      "price_clause": "str", "basis_price": "number", "tolerance": "str"}
RISK_SCHEMA        = {"risk_level": "enum[low,medium,high]", "credit_remaining": "number",
                      "blacklisted": "bool", "reasons": "list[str]"}
PRICING_SCHEMA      = {"deviation_pct": "number", "market_ref": "number", "conclusion": "str",
                      "reasons": "list[str]"}

WORKERS = {
    "extractor": {
        "system": ("你是点价评审小组的抽取员。只做合同要素抽取，逐字引用原文数字，"
                   "禁止推断或换算。输出必须严格是 JSON，字段按输出契约。"),
        "tools": [get_contract],
    },
    "risk": {
        "system": ("你是点价评审小组的风控员。只核查授信、敞口、黑名单与付款条款风险。"
                   "基于输入数据判断，不要重新查询行情。输出必须严格是 JSON。"),
        "tools": [get_credit],
    },
    "pricing": {
        "system": ("你是点价评审小组的定价员。只判断基准价与点价窗口的合理性，"
                   "计算偏离率保留两位小数。输出必须严格是 JSON。"),
        "tools": [get_market_price],
    },
}
```

```
# ----- 4. Worker 执行: 最小工具循环 + 轮次上限 -----
```

```
def tool_dispatch(worker: dict, name: str, args: dict) -> str:
    for fn in worker["tools"]:
        if fn.__name__ == name:
            return json.dumps(fn(**args), ensure_ascii=False)
    return json.dumps({"error": f"未知工具 {name}"}, ensure_ascii=False)

async def run_worker(role: str, handoff: Handoff) -> dict:
    """跑一个 Worker: system = 角色卡, user = handoff 四件套; 带工具循环与轮次上限"""
    worker = WORKERS[role]
    messages = [{"role": "system", "content": worker["system"]},
                {"role": "user", "content": handoff.render()}]
    for _ in range(MAX_TURNS):
        # 轮次上限: 防循环踢皮球
        resp = await client.chat.completions.create(
            model=MODEL_WORKER, messages=messages,
            tools=[{"type": "function",
                    "function": {"name": fn.__name__,
                                "description": fn.__doc__ or fn.__name__,
                                "parameters": {"type": "object", "properties": {}}}}
                  for fn in worker["tools"]])
        msg = resp.choices[0].message
        if not msg.tool_calls:
            # 没有工具调用 = 最终交付
            return {"role": role, "trace_id": handoff.trace_id,
                    "payload": json.loads(msg.content)}
        messages.append(msg)
        # 记录工具调用
        for tc in msg.tool_calls:
            # 执行并回填结果
            result = tool_dispatch(worker, tc.function.name, json.loads(tc.function.arguments or "{}"))
```

```

        messages.append({"role": "tool", "tool_call_id": tc.id, "content": result})
    raise RuntimeError(f"[{handoff.trace_id}] {role} 超过 {MAX_TURNS} 轮上限，熔断")

# ----- 5. 主管：固定委派 (L1) → 并行派发 → 校验 → 汇总 -----

def build_handoffs(trace_id: str, contract_no: str, customer: str) -> list[Handoff]:
    contract = get_contract(contract_no)          # 主管只发最小输入集合（上下文隔离）
    base = dict(trace_id=trace_id, from_role="lead")
    return [
        Handoff(**base, to_role="extractor",
            task="抽取该合同的全部评审要素",
            input_data={"contract_no": contract_no},
            output_schema=EXTRACTION_SCHEMA,
            boundary="禁止调用授信或行情工具，禁止推断"),
        Handoff(**base, to_role="risk",
            task="核查该客户与本笔交易的风险点",
            input_data={"customer": customer, "quantity_ton": contract["quantity_ton"],
                "basis_price": contract["basis_price"]},
            output_schema=RISK_SCHEMA,
            boundary="禁止查询行情，禁止给出定价建议"),
        Handoff(**base, to_role="pricing",
            task="评估基准价与点价窗口的合理性",
            input_data={"variety": contract["variety"],
                "settle_month": "2026-09", "basis_price": contract["basis_price"]},
            output_schema=PRICING_SCHEMA,
            boundary="禁止查询授信，禁止评价风险"),
    ]

def validate(payload: dict, schema: dict) -> list[str]:
    """交付物 Schema 校验：字段缺失 = 打回（生产中用 pydantic）"""
    missing = [k for k in schema if k not in payload]
    return [f"缺少字段: {k}" for k in missing]

async def run_lead(contract_no: str, customer: str) -> dict:
    trace_id = uuid.uuid4().hex[:8]
    print(f"[{trace_id}] 主管：拆解为 3 个子任务，并行派发")
    handoffs = build_handoffs(trace_id, contract_no, customer)

    results = await asyncio.gather(                # 并行派发（CompletableFuture.allOf 的 Python 版）
        run_worker("extractor", handoffs[0]),
        run_worker("risk", handoffs[1]),
        run_worker("pricing", handoffs[2]),
    )
    for r in results:
        errors = validate(r["payload"], {"extractor": EXTRACTION_SCHEMA,
            "risk": RISK_SCHEMA,
            "pricing": PRICING_SCHEMA}[r["role"]])
        if errors:
            raise ValueError(f"[{trace_id}] {r['role']} 交付不合规: {errors}")
    print(f"[{trace_id}] 三份交付物全部通过 Schema 校验，进入汇总")

prompt = ("你是点价评审小组主管。基于三份交付物交叉核对后出具评审意见。"
    "重点核对：抽取的数量与风控/定价使用的数量是否同一口径。\\n"
    + json.dumps([r["payload"] for r in results], ensure_ascii=False, indent=2))
resp = await client.chat.completions.create(model=MODEL_LEAD, messages=[
    {"role": "system", "content": "只输出 JSON: {conclusion, risk_points, pricing_advice,"

```

```
confidence}}",
    {"role": "user", "content": prompt}})
review = json.loads(resp.choices[0].message.content)
print(f"[{trace_id}] 评审意见: {json.dumps(review, ensure_ascii=False, indent=2)}")
return {"trace_id": trace_id, "review": review, "artifacts": [r["payload"] for r in results]}

if __name__ == "__main__":
    asyncio.run(run_lead(contract_no="CT-2026-0912", customer="华东铜业"))
```

2. 代码与概念的对应关系

代码元素	概念	Java 心智映射
Handoff 四件套	handoff 结构化交接	转派工单 DTO
WORKERS[role]["system"]	角色 Role/Persona	岗位 JD
asyncio.gather(...)	并行 SubAgent	CompletableFuture.allOf
MAX_TURNS	轮次上限熔断	循环边界 + 超时
validate(payload, schema)	交付物 Schema 前置校验	Bean Validation
trace_id 贯穿	可观测 trace	MDC / Sleuth 链路 ID
MODEL_WORKER vs MODEL_LEAD	模型分级路由	缓存分级 (本地/Redis)
run_worker 工具循环	单 Agent ReAct (004)	服务内 RPC 循环

3. 优先级要点与坑

1. **先跑通“双 Agent”**：把 WORKERS 砍到 1 个、gather 只留一项——先验证 handoff 与校验链路，再加 Worker；
2. **主管不做 LLM 拆解 (L1)**：拆解逻辑写死在 build_handoffs，把不确定性降到最低；要升级 L2（动态拆解）时，才让主管先输出子任务 JSON；
3. **Worker 的工具列表 = 角色边界**：tools 字段就是“只带 3~5 个专属工具”的落地，风控员物理上查不到行情；
4. **生产替换点**：get_* 模拟函数 → 006 讲的 MCP 工具调用；validate → pydantic 模型；print → 结构化日志上报 Langfuse/LangSmith；
5. **坑：JSON 解析崩溃**：Worker 输出夹带 markdown 代码块时 json.loads 会炸——生产中用 SDK 的结构化输出（response_format）或去掉围栏再解析。

4. 与框架的映射（细节见 07/009）

手写版	OpenAI Agents SDK	LangGraph
run_lead 编排	Handoff Agent 网络 / Supervisor	StateGraph + supervisor 节点

手写版	OpenAI Agents SDK	LangGraph
<code>Handoff</code> dataclass	handoff 参数传递	共享 State 的 reducer
<code>asyncio.gather</code>	并行子 Agent 原生支持	Send API 并发分支
手写工具循环	SDK 内置	预置 ReAct 图

【本节自测】

1. `Handoff` 四件套是什么？缺一个会怎样？ 要点：任务、输入数据、输出契约、边界；缺了就是"甩锅式交接"，Worker 靠猜。
2. `asyncio.gather` 对应 Java 什么原语？失败传播行为是什么？ 要点：`CompletableFuture.allOf`；任一 Worker 抛异常整体失败——生产中可 `return_exceptions=True` 收集后定向重试。
3. 为什么风控员"物理上"查不到行情？ 要点：角色工具集只挂 `get_credit`，工具边界即职责边界。
4. `MAX_TURNS` 防的是什么失败模式？ 要点：循环踢皮球/工具死循环（06 篇失败模式③）。
5. `validate` 打回的触发条件与动作？ 要点：交付物缺字段/不合 Schema；定向打回对应 Worker（代码里简化为报错，生产中最多回 1 轮）。
6. 升级到 L2 动态拆解要改哪里？ 要点：让主管 LLM 先输出子任务清单 JSON，`build_handoffs` 改为消费该清单，其余链路不变。
7. 最小双 Agent 版怎么裁剪？ 要点：WORKERS 留 1 个、gather 单项——先验证链路再扩编。

06 · 生产篇：成本、上下文与失败模式

【本节问题】① 多 Agent 系统在生产上最常见的翻车姿势是哪四种？② 每种失败模式的防御代码长什么样？③ 成本怎么预算、怎么护栏？④ 可观测性要记什么，出问题怎么回放？



1. 失败模式①：谁也不认账（交付物对不上）

症状：三个 Worker 各交各的——字段名随意（quantity / qty / 数量）、单位不统一（吨 vs 手）、缺关键字段；主管汇总时对齐口径，评审意见互相打架。**根因：**只派了任务，没定契约——“交付”没有验收标准。**防御（写进代码）：**1. **交付物 Schema 前置：**派活时 handoff 里就带 output_schema（05-实战篇的 Handoff.output_schema）；2. **自动校验 + 定向打回：**产出先过 pydantic/JSON Schema 校验，不合格附错误原因打回原 Worker，最多 1 轮；3. **公共事实字典：**跨 Worker 的共享事实（品种代码、单位、日期格式）维护一份字典，所有角色提示词引用同一份。

2. 失败模式②：信息丢失（handoff 变甩锅）

症状：交接消息只有“你去核查一下这个合同的风险”——不带数据不带口径，接手 Agent 只能重新瞎查，两次查询结果不一致，结论冲突。**根因：**把 handoff 当成了口头喊话，而不是材料移交。**防御：结构化 handoff 四件套必填——**任务目标 + 输入数据 + 输出 Schema + 边界（05-实战篇 Handoff dataclass 的四个字段，缺一不可，缺了直接抛异常而不是让 Agent 猜）。

3. 失败模式③：循环踢皮球（无限交接）

症状：A 说"这该 B 管", B 说"A 更清楚", handoff 打乒乓球；网状拓扑里没有拍板人，Token 白白烧掉直到有人手动杀进程。**根因：**没有全局轮次记账，没有终止仲裁者。**防御：**1. **全局 handoff 计数 + 轮次上限：**单任务总轮次封顶（点价评审建议 ≤ 6 ），超限熔断；2. **超时熔断：**单 Worker 墙钟超时（如 120s）即失败退出；3. **升级人工：**熔断后不是静默丢弃，而是带着已得的部分结论升级 HITL；4. **拓扑预防：**星型天然少发——拍板权始终在主管；用网状就必须显式设仲裁者。

```
# 防御③伪代码
class HandoffBudget:
    def __init__(self, max_hops: int = 6, timeout_s: int = 120):
        self.hops, self.max_hops, self.timeout_s = 0, max_hops, timeout_s
    def transfer(self, handoff: Handoff):
        self.hops += 1
        if self.hops > self.max_hops:
            raise EscalateToHuman(f"[{handoff.trace_id}] 交接超限，升级人工") # 带部分结论升级
```

4. 失败模式④：成本失控（Token × 15）

症状：并行多路 + 各自工具调用，消耗约为单 Agent 聊天的 15 倍（Anthropic 口径）；一次全量历史数据回溯跑批，月账单爆表。**根因：**把"多 Agent 不贵"当成了默认假设，没预算、没分级、没试跑。**防御：**1. **预算护栏：**单任务 Token 硬上限（如 $\leq 50k$ ）、单日团队上限（如 $\leq 2M$ ），超限拒绝开新任务；2. **模型分级路由：**Worker 用小杯（gpt-4o-mini 级），主管/汇总才用大杯——Worker 占 Token 大头，分级立省 60%+；3. **先抽样试跑：**上线前抽 20 笔真实单据试跑，实测均值 × 业务量 = 预算基线；4. **按价值分级启用：**Anthropic 的判断口径——任务价值足够高（付费研究、高风险评审）才付 15 倍的溢价，低价值查询留在单 Agent。

5. 可观测性：每条消息都是一笔流水

多 Agent 出问题时，"模型最终输出对不对"往往查不出来——要查**过程**。最小可观测四件套：

记录项	内容	用途
trace_id	全链路唯一 ID，handoff 间透传	串联一次评审的全部消息
角色与轮次	谁、第几轮、调了什么工具	定位"谁也不认账"发生在哪个环节
耗时与 Token	每条消息的墙钟时间 + prompt/completion Token	成本归因到 Worker 粒度
消息全文	handoff 与交付物原文（脱敏后）	事后回放定位口径冲突

落地：自建就打结构化日志（trace_id 进 MDC，Java 开发者零学习成本）；平台化接 **LangSmith / Langfuse**（均支持多 Agent trace 树）。回放姿势：按 trace_id 拉出整棵调用树 → 找到第一个口径不一致的消息 → 修 Schema 或提示词。

6. 生产检查清单（上线前逐条打勾）

- [] 每个 Worker 的交付物有 Schema 且有自动校验 + 打回机制
- [] Handoff 四件套必填，缺字段抛异常
- [] 全局轮次上限 + 单 Worker 超时熔断 + 升级人工通道
- [] 单任务/单日 Token 预算护栏 + 模型分级路由
- [] trace_id 全链路贯穿，消息全文可回放
- [] 用真实数据抽样试跑过，成本基线已测算
- [] 已回答"为什么不写成普通代码"（需要多视角 LLM 判断，而非规则可穷举）

【本节自测】

1. **四大失败模式分别是什么？** 要点：交付物对不上、handoff 信息丢失、循环踢皮球、成本失控（×15）。
2. **"定向打回"和"整体重跑"的区别？** 要点：只打回不合规的那个 Worker 并附错误原因（最多 1 轮），成本与延迟都远低于整体重跑。
3. **handoff 四件套缺一个的正确行为？** 要点：抛异常拒绝执行——让 Agent 猜 = 失败模式②的起点。
4. **为什么网状拓扑更需要轮次上限？** 要点：无拍板人、两两通信，乒乓循环风险最高；星型拍板权在主管。
5. **成本防御的四个动作？** 要点：预算护栏、模型分级、抽样试跑、按价值分级启用。
6. **最小可观测四件套？** 要点：trace_id、角色与轮次、耗时与 Token、消息全文；LangSmith/Langfuse 平台化。
7. **熔断后为什么必须升级人工而不是静默丢弃？** 要点：部分结论已花费成本，HITL 兜底保住业务连续性（CTRM 评审是高风险流程，静默失败不可接受）。
8. **检查清单里"为什么不写成普通代码"考什么？** 要点：防止 LLM 化过度——规则可穷举的步骤该写代码，Agent 只用于需要多视角判断的环节。

07 · 对比篇：框架与拓扑选型

【本节问题】① 拓扑怎么选，有没有可直接照走的决策树？② 主流多 Agent 框架各是什么定位，怎么选？③ 什么时候应该**坚持单 Agent**，谁有资格拍板？④ 这些框架和 009 篇怎么衔接？

1. 拓扑选型决策树

任务能否分解出边界清晰的独立子任务？

└ 否 → 坚持单 Agent（先优化 Prompt/工具/上下文压缩）

└ 是 → 子任务间有先后依赖（SOP 固定）？

└ 是 → 流水线（抽取→清洗→审核→归档）

└ 否 → 子任务结果需要汇总为单一结论？

└ 是 → 星型（主管 + Worker）★ 默认首选

└ 子任务规模大到一层拆不动？→ 层级（递归分治）

└ 否 → 真需要多方辩论/交叉审议？

└ 是 → 网状（必须配轮次上限 + 仲裁者）

└ 否 → 回到上一级重新审视（多半是星型）

口诀回顾（配 007-04）：**能流水线就流水线，要汇总用星型，大到拆不动用层级，真要辩论才上网状。** 点价评审走"是分解 → 无依赖 → 要汇总"分支 = **星型**。

2. 框架对比表（2026-09）

框架	核心抽象	强项	弱项	适用	现状笔记
LangGraph	StateGraph 状态图	图编排可控、checkpoint、HITL 原生 (interrupt)、LangSmith 观测成熟	学习曲线陡	企业生产、需审计与人工卡点	有测评称其 Token 效率比 CrewAI 高 15~18% (app-lab.ai 2026 综述引 shipsquad benchmark, 非官方)
OpenAI Agents SDK	Agent + handoff 原语	极简 API、免费内置 Tracing、guardrails	偏 OpenAI 生态	OpenAI 栈团队、客服路由、快速起步	Swarm 的生产化继任
CrewAI	crew + role/task	角色隐喻直观、20 分钟上手、已原生支持 A2A	灵活性较低	原型、内容/数据流水线	A2A 支持见 trendix 2026 报道
AutoGen / AG2	group chat 对话协作	多 Agent 辩论研究渊源深	配置复杂；有社区报道称 AutoGen 转入维护模式	辩论/验证类实验	截至 2026-09 未见微软官方文档宣布 EOL ；新项目社区建议 MAF 或 AG2

- 33 / 38 -

框架	核心抽象	强项	弱项	适用	现状笔记
Microsoft Agent Framework	Python + .NET 双栈	Azure 集成、企业治理、A2A GA	微软生态绑定	.NET 企业、Azure 客户	融合 AutoGen 与 Semantic Kernel 的官方新框架
Claude Managed Agents	云托管多 Agent	免运维	Anthropic 依赖	已在 Claude 栈、不想管基础设施	2026 年新发布的托管形态
手写 (05 篇)	代码即框架	零魔法、完全可控、教学价值最高	无checkpoint/重试等基建	学习原理、超简单场景	本章实战篇即此路线

选型三原则：1. **别按 GitHub 星数选**（多来源测评共识）——按"控制粒度需求 + 现有技术栈 + 可观测成熟度"选；2. **先框架后自研**：原型期用框架快速验证，框架成为硬约束时再抽关键件自研；3. **评测基建比框架重要**：框架差异 < 模型选择 + 评测体系（DeepEval 等）+ 人工卡点设计的影响。

3. "什么时候坚持单 Agent"裁定

裁定优先级（从上往下，命中任何一条就**不上**多 Agent）：

- 1. **规则可穷举**：步骤固定、无歧义 → 写代码/工作流引擎（Camunda 都比 Agent 便宜）；
- 2. **一次问答能解决**：单轮 + 少量工具调用搞定 → 单 Agent；
- 3. **子任务强耦合**：下游需要上游完整推理链 → 隔离反而制造断层；
- 4. **Token 预算 < 收益**：15 倍溢价买不回等值的时间/质量提升；
- 5. **合规要求全程单点可解释**：多 Agent 过程回放成本高于业务价值时。

反面裁定（满足 2 条以上，**该上**）：可分解、可并行、需不同专长/工具、上下文装不下、需交叉验证（见 01-问题篇信号清单）。

点价评审的最终裁定：满足 4 条该上信号（可分解/可并行/专长不同/需交叉验证），且是高风险高频核心流程，Token 溢价有业务价值兜底 → 上多 Agent，拓扑取星型，框架建议 LangGraph（需审计与 HITL）起步；评审链路固化后可降为流水线 + 局部 Agent 的混合形态。

4. 与 009 篇的衔接

- 本章 05 篇手写代码 = 框架抽象的"去糖"版本；009 篇用 LangGraph/OpenAI Agents SDK 把同一套点价评审重写为框架版，逐行对照抽象增益；
- 框架 API 的版本细节、部署与观测平台实操全部在 009 展开，本章不重复。

【本节自测】

- 1. **拓扑决策树的第一问是什么？** 要点："能否分解出边界清晰的独立子任务"——不能就坚持单 Agent。

2. **五种路线（含手写）各自的适用一句话？** 要点：LangGraph 企业可控；OpenAI SDK 轻量快速；CrewAI 快速原型；MAF Azure 企业；手写学与极简。
3. **AutoGen 的现状怎么表述才严谨？** 要点：有社区报道称转入维护模式，截至 2026-09 未见微软官方 EOL 宣告；新项目多建议 MAF/AG2。
4. **坚持单 Agent 的五条裁定？** 要点：规则可穷举、一次问答可解、强耦合、Token 不划算、单点可解释合规。
5. **选型三原则？** 要点：不按星数选；先框架后自研；评测基建比框架更重要。
6. **点价评审的完整裁定链？** 要点：4 条该上信号 → 星型 → LangGraph 起步 → 流程固化后可降级混合形态。
7. **CrewAI 在协议层面的亮点？** 要点：已原生支持 A2A，适合跨框架协作场景的原型验证。

08 • 自测篇：题库与面试题

【本节问题】① 前七节的要点我都能脱稿复述吗？② 概念题、场景题、面试题三个难度层我各能过几关？③ 面试官常设的坑在哪？④ 答不上来的点该回哪一节补？

A. 概念题（热身，答案见括号回指）

1. 多 Agent 的动机是什么？满足什么条件才值得拆？（回指 01：四大天花板 + 信号清单 2 条以上）
2. 角色三要素？（回指 02 卡 2：人设、边界、交付）
3. 点对点消息 vs 共享黑板怎么选？（回指 02 卡 4：委派审计 → 消息；共享事实 → 黑板 + Schema）
4. Handoff 的正确姿势与反面？（回指 02 卡 6：四件套移交 vs 甩锅）
5. 共享状态与私有上下文各管什么？（回指 02 卡 7：公共事实 vs 工作草稿）
6. A2A 三要素？（回指 02 卡 10：Agent Card / Task 状态机 / Artifact）
7. 15 倍 Token 谁说的、口径是什么？（回指 02 卡 11：Anthropic 2025-06，Agent≈4×、多 Agent≈15× 聊天）
8. 四种拓扑各配一个点价相关例子？（回指 04：评审=星型；抽取→清洗=流水线；全库盘点=层级；风险辩论=网状）

B. 场景题（实操，给思路要点）

1. 你负责 CTRM 的合同评审自动化，领导要求“全部 Agent 化”。你怎么裁？要点：先分拣——规则可穷举的步骤（金额换算、日期计算、状态流转）写代码；只有多视角判断环节（要素抽取、风险认定、价格评估）上 Agent；给出单 Agent 先行的证据再谈拆分。
2. 三个 Worker 的交付物字段名对不上，评审意见打架。排查与根治？要点：trace 回放定位第一个口径漂移点；根治 = 交付物 Schema 前置 + 自动校验打回 + 公共事实字典（06 篇失败模式①）。
3. 风控员和定价员的结论冲突（价格没问题但风险高）。主管怎么汇总？要点：这不是 bug 是交叉验证的价值；主管规则先行（风险高 → 评审结论降级为“暂缓”，无论定价如何），LLM 只做解释而非裁决；裁决规则要写进代码而不是指望模型。
4. 上线一周发现 Token 成本是预估的 3 倍。四步排查？要点：按 trace 归因到 Worker 粒度 → 查是否有人传完整历史而非结构化产物 → 查模型分级是否失效 → 补预算护栏 + 抽样重估基线（06 篇失败模式④）。
5. 业务方要求接入外部供应商的报价 Agent。用什么协议接入，为什么不用 MCP 工具？要点：对方是自主 Agent、任务长、可能中途澄清 → A2A；先拉 Agent Card 验能力与鉴权（v1.0 JWS 签名卡）；MCP 工具适合固定输入输出的一次性调用（04 篇灰区判断）。
6. 评审小组要把“主管拆解”从写死改为 LLM 动态拆解（L1→L2），风险与前置条件？要点：风险 = 拆解错误传导全链；前置 = 主管输出子任务 JSON 也过 Schema 校验 + 全局轮次上限 + 打回预算，再灰度放量（03/06 篇）。

C. 面试题（如实战追问，答出“结构 + 依据”才算过关）

1. “为什么要多 Agent？单 Agent 加长上下文不行吗？” 参考结构：结构性天花板（装不下/选不清/串行/混杂）→ context rot 证据（003）→ 但先亮代价（15×）→ 给裁定信号清单。踩坑警告：只吹好处不提 15 倍与“能单就单”，会被认为没上过生产。
2. “设计一个点价评审的多 Agent 系统，现场画架构。” 参考结构：星型拓扑（配 007-06）→ 主管/抽取/风控/定价职责与工具边界 → handoff 四件套 → Schema 校验打回 → 轮次上限与 HITL → trace 与预算。加分点：主动说“拆解逻辑写死（L1）降低不确定性”。
3. “A2A 和 MCP 什么关系？有了 MCP 为什么还要 A2A？” 参考结构：两层互补（连工具 vs 连 Agent）→ 工作单元差异（单次调用 vs Task 状态机）→ 暴露差异（显式 schema vs 不透明）→ 治理现状（2026-08 起同属 AAIF）→ 灰区判断（自主性/多轮/长任务选 A2A）。加分点：提到 MCP 实验性 tasks 原语但不混淆本质。
4. “多 Agent 系统 Token 为什么放大 15 倍？怎么控？” 参考结构：放大来源（Worker 重复读背景/交接载荷/编排消耗/并行）→ 四个防御（预算护栏/模型分级/抽样试跑/按价值分级）→ Anthropic 出处。
5. “Worker 之间该共享多少上下文？” 参考结构：默认最小集合（隔离收益）→ 共享的只有公共事实字典与交付物 → 反例（强耦合任务隔离反而制造断层）→ 结论：按“事实类型”分层而不是“全共享或全私有”。
6. “你的多 Agent 上线后卡死了，怎么建可观测？” 参考结构：trace_id 全链路 → 消息级流水（角色/轮次/耗时/Token）→ LangSmith/Langfuse 平台化 → 回放定位口径漂移点 → 把修复固化回 Schema/提示词。

D. 快问快答（30 秒内）

问	答
星型拓扑的瓶颈？	主管：上下文与吞吐；Worker 建议 3~5 个
网状拓扑的第一护栏？	全局轮次上限 + 显式仲裁者
handoff 缺字段怎么办？	抛异常拒绝执行，不让 Agent 猜
打回重做的上限？	定向打回最多 1 轮
A2A 的发现端点？	<code>/well-known/agent-card.json</code>
Task 卡在哪个状态说明要人介入？	input-required
进程内 Worker 之间该走 A2A 吗？	不该，直接调用（agents-as-tools inside a crew）
评审系统第一版拆解逻辑放哪？	写死在代码（L1），不放 LLM

【本节自测】

1. 三个难度层各过关标准？ 要点：概念题脱稿、场景题给“排查+根治”两段、面试题给“结构+依据+出处”。

2. **最容易翻车的面试题是哪道？** 要点：第 1 题——不提 15 倍代价与"能单就单"即失分。
3. **A2A 面试题的加分点？** 要点：知道 MCP tasks 实验原语但能说清两者本质差异。
4. **场景题 3 的核心原则？** 要点：裁决规则写进代码，LLM 只解释不裁决。
5. **全部答不出来时的复习路径？** 要点：01 天花板与信号 → 02 概念卡 → 04 拓扑与 A2A → 06 失败模式与防御 → 05 代码对照。